Title: LED Blinking and Time Display - 10-Second Interval Control

Name: MD SHAKIL KHAN
Student ID: 2330130220
Date of Lab: October 27, 2025

Introduction-

Before this lab, it was known that LED blinking is a fundamental embedded systems task used to verify microcontroller operation and timing control. The lab is about implementing precise timing control to make an LED blink at specific intervals while simultaneously displaying elapsed time through serial communication. The purpose of this lab was to modify existing code to create a system where an LED blinks every 10 seconds while the elapsed time is displayed through UART serial communication, demonstrating the integration of hardware output and software timing.

Materials-

Apparatus	Expendables & Software
• Development Board with onboard LED	• AHL GEC IDE v4.53
• USB Cable	• Example Project File
• PC/Laptop	• Serial Terminal Software (PuTTY/Tera Term)
• Power Supply	• USB Drivers

Procedure-

The procedure was followed as outlined in the lab handout. The key steps involved:

1) Opening the LED control project in AHL GEC IDE v4.53.
2) Reviewing the main.c source code to understand LED configuration and timing functions.
3) Compiling and downloading the initial HEX file to verify basic functionality.
4) Modifying the code to implement a 10-second delay between LED state changes.
5) Adding UART functionality to transmit elapsed time information after each LED toggle.

6) Recompiling the modified program and flashing it to the development board.
7) Verifying both LED blinking behavior and serial time display operated correctly at 10-second intervals.

Data-

Observed LED Behavior:

- LED turned ON and OFF alternately every 10 seconds
- The timing remained consistent throughout operation
- No visible timing drift or missed intervals observed

Serial Terminal Output:

Time elapsed: 10 seconds

Time elapsed: 20 seconds

Time elapsed: 30 seconds

Time elapsed: 40 seconds

...

Variables Used in the Project:

- seconds: Accumulated time counter (incremented by 10 each cycle)
- buffer[50]: Character array for formatting the time display message
- LED_Toggle(): Function to switch LED state between ON and OFF
- UART_SendString(): Function to transmit data through serial communication
- Delay(10000): Function creating 10,000 millisecond (10-second) pause

Calculations-

The primary calculation involved timing configuration:

- Desired delay: 10 seconds = 10,000 milliseconds
- Delay function parameter: Delay(10000)
- Time increment per cycle: 10 seconds

Code Implementation:

```
#include "UART.h"
#include "Delay.h"
#include "LED.h"

int main(void)
{
unsigned int seconds = 0;
UART_Init(); // Initialize UART for serial output
LED_Init();  // Configure LED output pin

while(1)
{
LED_Toggle(); // Change LED state (ON/OFF)
seconds += 10; // Increment time counter by 10 seconds

// Display elapsed time through UART
char buffer[50];
sprintf(buffer, "Time elapsed: %d seconds\r\n", seconds);
UART_SendString(buffer);

Delay(10000); // 10-second delay
}
}
```

Analysis

1. Why was a 10-second delay chosen instead of a shorter interval?
A 10-second delay was used because it provides easily observable timing for verification purposes. Shorter intervals (like 1 second) would make it difficult to distinguish precise timing, while the 10-second gap allows clear observation of both the LED behavior and serial output synchronization.

2. What could cause timing inaccuracies in this system?
Timing inaccuracies could result from an improperly calibrated system clock, interrupt conflicts that delay the main program execution, or inaccuracies in the delay function itself. If the microcontroller's clock frequency isn't precisely configured, the actual delay might be slightly longer or shorter than 10 seconds.

Conclusions

The purpose of this lab was to create a system that blinks an LED every 10 seconds while displaying elapsed time through serial communication. This was successfully accomplished; the LED blinked at consistent 10-second intervals, and the serial terminal displayed corresponding time messages (10, 20, 30 seconds, etc.) in perfect synchronization with the LED changes.

The experiment demonstrated high precision with no observable timing drift. Potential sources of error could include system clock miscalibration or interrupt handling conflicts that might disrupt the timing loop. However, these were not encountered during this implementation.

This exercise relates directly to real-world applications such as status indicators in industrial monitoring systems, periodic data logging in environmental sensors, heartbeat signals in network devices, and timing controls in automation systems. The ability to synchronize hardware outputs with software timing is fundamental to embedded systems used in IoT devices, industrial controls, and consumer electronics.